

NetSentinel AI

Plateforme de détection d'intrusion assistée par intelligence artificielle —
Documentation technique complète

Projet de soutenance — Licence 3 ICT

13 juillet 2026

Table des matières

1	Résumé exécutif	3
1.1	État réel au 13 juillet 2026	3
2	Contexte et objectifs	4
2.1	Le problème traité	4
2.2	Objectifs du projet	4
2.3	Cas d'usage couverts	4
3	Architecture générale	5
3.1	Vue d'ensemble	5
3.2	Principe de conception : la séparation détection / décision	5
3.3	Rôle de chaque composant	6
4	Les agents de collecte	7
4.1	Les trois sondes	7
4.2	Le flux d'enrôlement en quatre états	7
4.3	Jetons d'enrôlement	7
4.4	Installation	7
4.5	Métadonnées d'inventaire	8
5	Le stockage : Elasticsearch	9
5.1	Configuration	9
5.2	Index utilisés	9
5.3	Un point d'attention : la volumétrie	9
6	Le backend	11
6.1	Rôle	11
6.2	Un backend à stockage interchangeable	11
6.3	Le calcul du score de risque	11
6.4	Corrélation en incidents	12
7	Le moteur de détection IA	13
7.1	Le cycle de détection en 8 étapes	13
7.2	Étape 3 — Les features calculées	13

7.3	Étape 4 — Les cinq heuristiques	13
7.4	Étape 5a — IsolationForest (non supervisé)	14
7.5	Étape 5b — RandomForest (supervisé)	14
7.5.1	Honnêteté sur l'entraînement	15
7.6	Étape 6 — La déduplication	15
7.7	Étape 8 — La prévention	16
8	Le frontend	17
9	Déploiement en production	18
9.1	Le serveur	18
9.2	Les services systemd	18
9.3	Le timer de détection	19
10	Journal des corrections — 13 juillet 2026	20
10.1	Le symptôme	20
10.2	Trois défauts en cascade, chacun masquant le suivant	20
10.2.1	Défaut 1 — Mauvais port Elasticsearch	20
10.2.2	Défaut 2 — Chemin d'état inexistant	20
10.2.3	Défaut 3 — Régression de refactoring, masquée par une erreur silen- cieuse	20
10.3	Vérification après correction	21
10.4	Corrections livrées	21
11	Limites connues	22
11.1	Les deux priorités avant la soutenance	23
12	Ce qu'il faut défendre en soutenance	24
12.1	Les points forts	24
12.2	Les questions probables du jury — et les réponses	24
12.3	Le déroulé de démonstration recommandé	25
13	Annexes	26
13.1	Annexe A — Référence complète de l'API (42 routes)	26
13.2	Annexe B — Variables d'environnement	27
13.3	Annexe C — Arborescence du projet	28
13.4	Annexe D — Les alertes réelles du 13 juillet 2026	29
13.5	Annexe E — Environnement technique	29

1 Résumé exécutif

NetSentinel AI est une plateforme de supervision de sécurité (SOC) qui transforme des événements techniques bruts — journaux système, flux réseau, métriques machine — en informations exploitables : alertes qualifiées, incidents corrélés, scores de risque par machine et recommandations de remédiation.

La plateforme couvre la chaîne complète :

Collecte → Stockage → Analyse → Visualisation → Réponse

Elle se distingue d'un simple tableau de bord de logs par son **moteur de détection hybride**, qui combine trois familles de détecteurs complémentaires :

1. **Heuristiques explicables** (5 détecteurs à base de règles) — logique transparente, défendable ligne à ligne ;
2. **Apprentissage non supervisé** (IsolationForest) — détecte l'anormal *inconnu*, sans étiquetage préalable ;
3. **Apprentissage supervisé** (RandomForest) — classe les types d'attaque *connus* (brute force, scan, déni de service, escalade de privilèges).

Chaque détection est rattachée à une **tactique MITRE ATT&CK**, ce qui inscrit la plateforme dans un référentiel professionnel reconnu.

1.1 État réel au 13 juillet 2026

La plateforme est **déployée et opérationnelle** sur un serveur de production (VPS 79.137.32.27), où elle analyse en continu la télémétrie d'un hôte réel exposé sur Internet.

Indicateur	Valeur mesurée
Journaux système collectés (Filebeat)	5 103 658 documents (1,2 Go)
Métriques machine (Metricbeat)	41 061 889 documents (27,3 Go)
Flux réseau (Packetbeat)	34 802 documents (36 Mo)
Alertes de sécurité actives	4
Incidents corrélés	1 (INC-00001)
Hôtes supervisés	1 (vps-8e515079)
Cycle de détection	automatique, toutes les 5 minutes

Les 4 alertes actives ne sont **pas des données de démonstration** : ce sont de véritables tentatives d'intrusion détectées sur le serveur, provenant des adresses 135.181.107.18 et 45.148.10.239, qualifiées en *Credential Access* (MITRE ATT&CK) par deux détecteurs indépendants — l'heuristique SSH et le Random Forest — qui **convergent sur le même verdict**.

2 Contexte et objectifs

2.1 Le problème traité

Une machine exposée sur Internet est attaquée en permanence. Les journaux du système enregistrent ces tentatives, mais sous une forme inexploitable : des millions de lignes indifférenciées, où l'attaque réelle est noyée dans le bruit de fonctionnement normal.

Le serveur de ce projet en est l'illustration directe : il a produit plus de **46 millions d'événements** en trois semaines. Aucun opérateur humain ne peut lire cela.

Le problème n'est donc pas de *collecter* — c'est de **décider** : parmi des millions d'événements, lesquels méritent l'attention d'un analyste ?

2.2 Objectifs du projet

Objectif	Réponse apportée
Centraliser les journaux d'un parc de machines	Agents Beats → Elasticsearch
Détecter les comportements suspects	Moteur hybride heuristiques + ML
Réduire le bruit	Déduplication par signature, fenêtre de 60 min
Rendre les alertes exploitables	Corrélation en incidents, recommandations
Rattacher à un référentiel	MITRE ATT&CK (tactiques)
Visualiser l'état de sécurité	Interface web orientée SOC (18 pages)
Déployer de façon contrôlée	Agent installable, flux d'enrôlement à 4 états

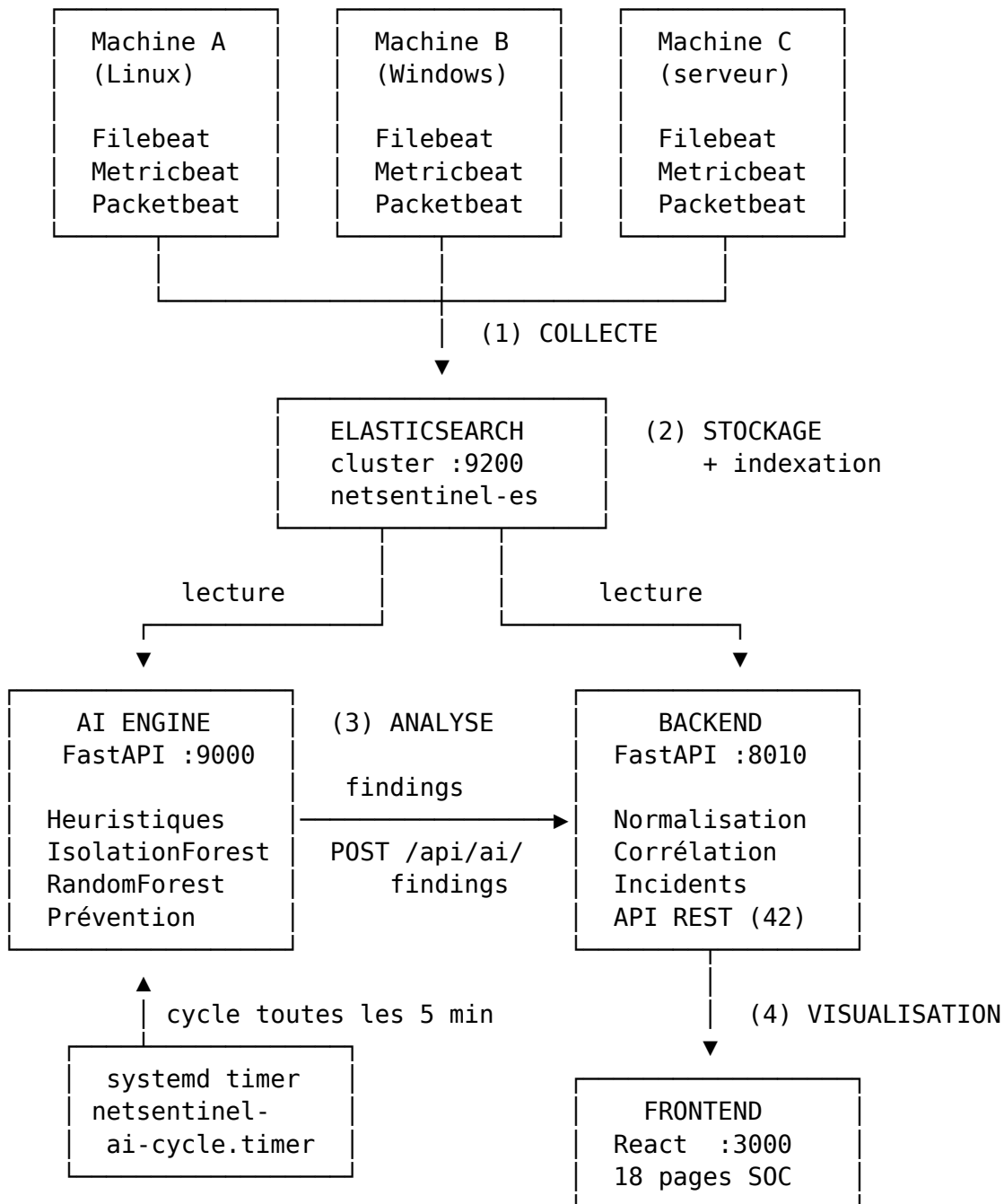
2.3 Cas d'usage couverts

- Tentative de **force brute SSH** (détectée en production) ;
- **Balayage de ports** (*port scan*) ;
- **Activité DNS anormale** (exfiltration, canal de commande) ;
- **Escalade de privilèges** (usage anormal de sudo) ;
- **Déplacement latéral** entre machines internes ;
- **Anomalies inconnues** ne correspondant à aucune règle écrite.

3 Architecture générale

3.1 Vue d'ensemble

La plateforme repose sur **cinq composants** communiquant par HTTP, chacun isolé dans son propre processus.



3.2 Principe de conception : la séparation détection / décision

Un choix d'architecture structurant mérite d'être souligné, car il est défendable devant un jury :

Les détecteurs ne bloquent jamais directement.

Le moteur IA *détecte* et produit des *findings*. Toute décision de blocage passe par un module dédié (`prevention.py`), lui-même appelant une route du backend. Cette séparation permet de :

- **auditer** toutes les décisions de blocage en un seul point ;
- **désactiver** la réponse automatique sans toucher à la détection ;
- **protéger** des adresses critiques par une liste blanche (NEVER_BLOCK), garde-fou qui empêche la plateforme de se couper elle-même du réseau.

3.3 Rôle de chaque composant

Composant	Techno	Port	Responsabilité
Agents	Elastic Beats	—	Collecter logs, métriques, flux réseau
Elasticsearch	ES 9.4.2	9200	Stocker et indexer les événements
AI Engine	FastAPI / Python 3.13	9000	Détecter les menaces, décider des blocages
Backend	FastAPI / Python 3.13	8010	Normaliser, corréler, exposer l'API REST
Frontend	React 18 / Node 20	3000	Interface SOC

4 Les agents de collecte

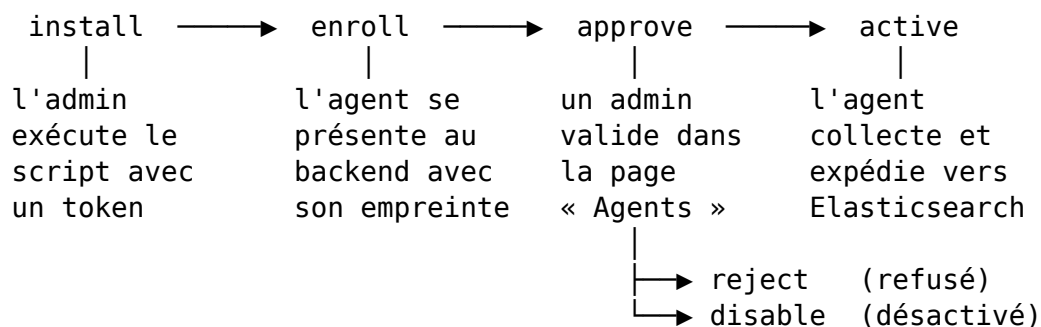
4.1 Les trois sondes

Chaque machine supervisée héberge trois sondes Elastic Beats, complémentaires :

Sonde	Ce qu'elle observe	Exemple d'usage en détection
Filebeat	Journaux système (/var/log/auth.log...)	Échecs d'authentification SSH, appels sudo
Metricbeat	CPU, mémoire, disque, réseau	Score de risque de l'hôte, anomalies de charge
Packetbeat	Flux réseau (DNS, HTTP, TCP)	Balayage de ports, requêtes DNS anormales

4.2 Le flux d'enrôlement en quatre états

Un agent ne collecte **rien** tant qu'il n'a pas été explicitement approuvé côté serveur. C'est un point de sécurité important : distribuer un installateur ne revient pas à ouvrir son réseau.



Les états sont pilotés depuis l'interface (page **Agents**) ou par API :

- POST /api/agent/enroll — l'agent se déclare ;
- POST /api/agent/instances/{id}/approve — l'admin autorise la collecte ;
- POST /api/agent/instances/{id}/reject — l'admin refuse ;
- POST /api/agent/instances/{id}/disable — l'admin coupe un agent actif ;
- POST /api/agent/heartbeat — l'agent signale qu'il est vivant ;
- POST /api/agent/checkin — l'agent récupère sa configuration.

4.3 Jetons d'enrôlement

L'installation exige un **jeton** (*enrollment token*), créé depuis la page Agents ou via POST /api/agent/enrollment-tokens, et révocable à tout moment (.../revoke). Sans jeton valide, aucun agent ne peut se déclarer.

4.4 Installation

Linux (Ubuntu) :

```
TOKEN="<jeton>"; API_URL="http://79.137.32.27:8010"
TMP_DIR="$(mktemp -d)" \
  && curl -fsSL "https://raw.githubusercontent.com/eliote-geeks/\
soutenance_ict_l3/main/agent/install-linux.sh" -o "$TMP_DIR/install-linux.sh" \
  && sudo bash "$TMP_DIR/install-linux.sh" \
    --api-url "$API_URL" --enrollment-token "$TOKEN"
```

Windows (PowerShell administrateur) : script `install-windows.ps1`, même logique.

Des paquets natifs sont également prévus : `packaging/linux/build-deb.sh` (paquet `.deb`) et `packaging/windows/build-installer.ps1`.

4.5 Métadonnées d'inventaire

Chaque agent porte une identité métier, exploitée pour filtrer et prioriser :

Champ	Rôle	Valeur observée en production
<code>asset_id</code>	Identifiant unique de l'actif	<code>asset_paul_serveur_1781781858100</code>
<code>profile_id</code>	Profil de rattachement	<code>profile_lab</code>
<code>role</code>	Fonction de la machine	<code>workstation</code>
<code>site</code>	Localisation	<code>yaounde-lab</code>
<code>environment</code>	Environnement	<code>lab</code>

5 Le stockage : Elasticsearch

5.1 Configuration

Paramètre	Valeur
Version	Elasticsearch 9.4.2
Nom du cluster	netsentinel-es
Nœuds	1 (nœud unique)
État	yellow
Port	9200
Mémoire JVM	1 Go (-Xms1g -Xmx1g)

Note sur l'état yellow : il est **normal et attendu** sur un cluster à nœud unique. Elasticsearch demande une réplique de chaque *shard* primaire, mais refuse de la placer sur le même nœud (une réplique sur la même machine ne protège de rien). Les *shards* répliques restent donc non assignés, d'où le yellow. Les données sont intègres et complètes. Passer en green exigerait un second nœud — hors périmètre pour un projet de laboratoire. **C'est une question classique de jury : la bonne réponse est « yellow ≠ dégradé ».**

5.2 Index utilisés

Index	Contenu	Documents	Taille
.ds-filebeat-8.19.16-*	Journaux système	5 103 658	1 167 Mo
.ds-metricbeat-8.19.16-*	Métriques machine	41 061 889	27 297 Mo
.ds-packetbeat-8.19.16-*	Flux réseau	34 802	36 Mo
ai-alerts-manual	Findings produits par l'IA	4	< 1 Mo
netsentinel-profiles	Profils métier	—	—
netsentinel-assets	Inventaire des actifs	—	—
netsentinel-agent-instances	Agents enrôlés	—	—
netsentinel-agent-enrollment-tokens	Jetons d'enrôlement	—	—

Les trois index Beats sont des **data streams** (flux de données), le mécanisme Elasticsearch adapté aux séries temporelles : écriture continue en append, rotation automatique des index sous-jacents.

5.3 Un point d'attention : la volumétrie

Metricbeat produit à lui seul **27,3 Go**, soit **~1,1 Go/jour**. Le disque du serveur est occupé à **81 %** (56 Go libres), ce qui laisse environ **50 jours** avant saturation.

Ce n'est pas un problème *aujourd'hui*, mais c'est une dette d'exploitation réelle. Deux remédiations possibles :

1. **Politique ILM de rétention** — supprimer automatiquement les données de plus de 30 jours ;
2. **Réduire la fréquence de collecte** de Metricbeat (period: 10s → 60s), qui divise le volume par six.

6 Le backend

6.1 Rôle

Le backend (FastAPI, port **8010**) est la couche métier de la plateforme. Il :

- lit Elasticsearch et **normalise** les données brutes en objets métier ;
- **corrèle** les alertes en incidents ;
- **calcule** les scores de risque par hôte ;
- **expose** 42 routes REST consommées par le frontend ;
- **gère** le cycle de vie des agents (enrôlement, approbation) ;
- **reçoit** les findings du moteur IA (POST /api/ai/findings).

6.2 Un backend à stockage interchangeable

Choix d'architecture notable : le backend **n'est pas marié à Elasticsearch**. Deux axes sont configurables indépendamment.

Stockage applicatif (NETSENTINEL_STORAGE_BACKEND) — profils, actifs, agents, findings :

Valeur	Comportement
elastic	Stockage dans Elasticsearch (<i>configuration de production</i>)
json	Stockage dans un fichier local (STORAGE_JSON_PATH)
postgresql	Stockage en base relationnelle (DATABASE_URL)
demo	Jeux de données de démonstration, sans persistance

Télémétrie de sécurité (NETSENTINEL_TELEMETRY_BACKEND) — logs, flux, métriques :

Valeur	Comportement
elastic	Lecture des index Beats (<i>configuration de production</i>)
json	Lecture d'un fichier JSON de télémétrie
demo	Jeux de données de démonstration

Cette séparation permet de faire tourner l'application **sans Elasticsearch** (utile en développement ou pour une démonstration hors ligne), tout en conservant Elasticsearch en production.

6.3 Le calcul du score de risque

Le score de risque d'un hôte est calculé à partir des métriques Metricbeat réelles :

```
risk = min(95, int((cpu_ratio * 45) + (mem_ratio * 35) + 20))
```

- une base de **20** (aucune machine n'est à risque nul) ;

- la charge CPU pèse jusqu'à **45 points** ;
- l'occupation mémoire jusqu'à **35 points** ;
- le score est plafonné à **95** (aucune certitude absolue).

Un hôte au-dessus de **70** est classé high, sinon medium.

6.4 Corrélation en incidents

Le backend regroupe les alertes partageant une même **tactique MITRE** et une même **source** en un incident unique (INC-00001, INC-00002...). Un incident porte le nombre d'alertes agrégées et le nombre d'hôtes affectés — c'est ce qui distingue un flot d'alertes d'une véritable histoire d'attaque.

En production, les 4 alertes se sont regroupées en **1 incident** : « *Credential Access from 45.148.10.239* », sévérité critical.

7 Le moteur de détection IA

C'est le cœur du projet. Le moteur IA **n'est pas un chatbot** : c'est un service d'analyse de sécurité.

7.1 Le cycle de détection en 8 étapes

Le cycle (`ai_engine/cycle.py`) est déclenché toutes les 5 minutes par un *timer* systemd et enchaîne :

1. Lecture de la fenêtre courante — Filebeat + Packetbeat (10 dernières min)
2. Lecture de l'historique — 24 h, en fenêtres de 15 min (base ML)
3. Construction des features — agrégation par adresse IP source
4. Détecteurs heuristiques (5) — règles explicables
5. Détecteurs ML (2) — IsolationForest + RandomForest
6. Déduplication — signature SHA-256, fenêtre 60 min
7. Publication — POST `/api/ai/findings` vers le backend
8. Prévention — blocage des IP de sévérité critique

7.2 Étape 3 — Les features calculées

Les événements bruts sont agrégés **par adresse IP source** en un vecteur de 8 caractéristiques. C'est cette transformation qui rend l'apprentissage automatique possible :

Feature	Signification	Signal d'attaque associé
<code>event_count</code>	Volume d'événements	Déni de service
<code>is_internal</code>	Source interne ou externe	Déplacement latéral
<code>failed_logins</code>	Échecs d'authentification	Force brute
<code>dns_errors</code>	Erreurs DNS	Canal de commande, exfiltration
<code>distinct_ports</code>	Diversité des ports visés	Balayage de ports
<code>distinct_destinations</code>	Diversité des destinations	Reconnaissance
<code>protocol_count</code>	Diversité des protocoles	Comportement atypique
<code>http_path_count</code>	Diversité des chemins HTTP	Fuzzing web

7.3 Étape 4 — Les cinq heuristiques

Détecteurs à base de règles, dont la logique est **entièrement explicable** — un atout majeur en soutenance : chaque alerte peut être justifiée ligne à ligne.

Détecteur	Règle	Seuil	Tactique MITRE
<code>detect_ssh_bruteforce</code>	Trop d'échecs SSH sur une courte fenêtre	5 échecs	Credential Access
<code>detect_dns_anomaly</code>	Rafale d'erreurs DNS	8 erreurs	Command and Control

Détecteur	Règle	Seuil	Tactique MITRE
detect_port_scan	Trop de ports distincts visés	8 ports	Discovery
detect_privilege_escalation	Motifs sudo / élévation anormaux	—	Privilege Escalation
detect_lateral_movement	Source interne visant plusieurs machines	—	Lateral Movement

La **confiance** d'un finding est dérivée du dépassement du seuil (`confidence_from_ratio`), puis la **sévérité** est dérivée de la confiance (`severity_from_confidence`) : plus le signal dépasse le seuil, plus l'alerte est grave. Le lien entre l'observation et la gravité est donc traçable.

7.4 Étape 5a — IsolationForest (non supervisé)

Objectif : détecter l'anomalie *inconnue* — celle qu'aucune règle écrite ne prévoit.

Paramètre	Valeur
Algorithme	IsolationForest (scikit-learn)
Arbres	200 (<code>n_estimators=200</code>)
Contamination	0,12 (12 % d'outliers attendus)
Normalisation	StandardScaler
Historique de référence	24 h, fenêtres de 15 min
Échantillons minimum	10
Graine aléatoire	42 (résultats reproductibles)

Principe : l'algorithme construit une forêt d'arbres qui *isolent* les points. Un point anormal, par définition éloigné des autres, est isolé en **peu de coupures** — c'est ce faible nombre de coupures qui le trahit. Aucun étiquetage n'est nécessaire : le modèle apprend seul ce qu'est le trafic « normal » de ce réseau, à partir de son historique.

Force : détecte le jamais-vu. **Limite :** dit « ceci est anormal » sans dire *pourquoi* — d'où la complémentarité avec les heuristiques.

7.5 Étape 5b — RandomForest (supervisé)

Objectif : classer les types d'attaque *connus*.

Paramètre	Valeur
Algorithme	RandomForestClassifier (scikit-learn)
Features en entrée	Les 8 features agrégées
Normalisation	StandardScaler
Modèle persisté	<code>ai_engine/state/models/</code>

Classes prédites et correspondance MITRE :

Classe	Tactique MITRE	Sévérité attribuée
bruteforce	Credential Access	critical
probe	Discovery	high
dos	Impact	critical
privilege_escalation	Privilege Escalation	critical
other	Execution	medium

7.5.1 Honnêteté sur l'entraînement

Le modèle est entraîné (`train_random_forest.py`) sur un **jeu de données synthétique** généré :

Classe	Échantillons
Trafic normal	400
Force brute	200
Balayage de ports	200
Déni de service	200
Escalade de privilèges	150
Total	1 150

Ce point sera challengé par le jury — il faut l'assumer de front. Un jeu synthétique n'est pas un jeu réel : le modèle apprend les frontières que le générateur a dessinées, et ses performances mesurées sur ce jeu seraient optimistes.

La défense est solide, en trois temps :

1. **C'est un choix contraint et documenté.** Il n'existe pas de jeu étiqueté du trafic *de ce réseau précis* ; étiqueter manuellement 46 millions d'événements est hors de portée d'un projet de licence.
2. **Le RF n'est pas seul.** Il constitue une des trois familles de détecteurs. Les heuristiques ne dépendent d'aucun apprentissage, et l'IsolationForest apprend, lui, sur les **données réelles** du réseau. Une faiblesse du RF ne compromet donc pas la détection.
3. **La validation est venue du terrain.** Sur les attaques réelles observées en production, le Random Forest et l'heuristique SSH — **deux méthodes totalement indépendantes** — ont convergé sur le même verdict (bruteforce / *Credential Access*) pour les mêmes adresses IP. Cette concordance sur des données non synthétiques est le meilleur argument disponible.

Évolution naturelle : réentraîner le modèle sur les findings confirmés par l'analyste, qui constituent progressivement un jeu réel étiqueté. C'est la boucle d'amélioration continue à présenter comme perspective.

7.6 Étape 6 — La déduplication

Sans déduplication, une attaque en cours génère une alerte identique à chaque cycle, soit 12 par heure : l'analyste est noyé, et la plateforme devient inutilisable.

Mécanisme : chaque finding reçoit une **signature SHA-256** calculée sur le tuple :

titre | ip_source | ip_destination | hostname | tactique_mitre

Si une signature identique a déjà été publiée dans les **60 dernières minutes** (FINDING_SUPPRESSION_MINUTES), le finding est supprimé. L'état est persisté sur disque (ai_engine/state/finding_state.json) et purgé au-delà de 7 jours.

*Vérification en production : le premier cycle a publié 4 findings ; le cycle suivant, 5 minutes plus tard, a de nouveau détecté les 4 mêmes menaces mais n'en a publié **aucune** — la déduplication fonctionne exactement comme prévu.*

7.7 Étape 8 — La prévention

Les findings de sévérité **critique** déclenchent une demande de blocage de l'IP source, via POST /api/firewall/block.

Garde-fou : la liste NEVER_BLOCK (127.0.0.1, ::1, localhost) empêche la plateforme de se bloquer elle-même.

En production, 3 adresses ont été soumises au blocage : 135.181.107.18, 45.148.10.239, 77.90.185.20.

Limite à assumer explicitement (voir chapitre « Limites connues ») : dans l'état actuel, /api/firewall/block **n'applique aucune règle iptables**. Il ajoute l'adresse à un ensemble en mémoire. Le blocage est donc **logique** (tracé, affiché, auditable) et non **effectif** (le paquet réseau n'est pas rejeté). Il ne faut en aucun cas prétendre le contraire devant le jury.

8 Le frontend

Interface React (port **3000**), orientée poste d'analyste SOC. **18 pages** :

Page	Fonction
Overview	Tableau de bord principal, indicateurs de sécurité
Alerts	Liste des alertes, filtres, acquittement
Incidents	Incidents corrélés, hôtes affectés
Hosts	Parc supervisé, scores de risque, isolation
Agents	Enrôlement, approbation, rejet, désactivation
Logs Explorer	Exploration des journaux bruts
Live Stream	Flux d'événements en temps réel
Network Map	Cartographie réseau
Model	État et paramètres des modèles IA
Predictions	Prédictions du moteur
Pipeline	Santé de la chaîne de traitement
Reports	Export de rapports
Users / Profile / Settings	Administration
Setup	Assistant de première configuration
Reset	Réinitialisation de l'application
User Guide	Documentation intégrée

Technologies : React, CRACO, Tailwind CSS, composants Radix UI, Recharts (graphiques). En production, le build statique est servi par frontend/server.js (Node 20).

Deux composants sont dédiés à la présentation : LaunchIntro (écran d'introduction) et PresentationSlides (support de soutenance intégré à l'application).

9 Déploiement en production

9.1 Le serveur

Caractéristique	Valeur
Adresse	79.137.32.27
Nom d'hôte	vps-64194ecd
CPU	12 vCPU
Mémoire	45 Go
Disque	290 Go (81 % occupé)
Système	Ubuntu Linux
Accès	ssh ubuntu@79.137.32.27

9.2 Les services systemd

Quatre services permanents et un *timer* :

Unité	Rôle	Port
netsentinel-elasticsearch.service	Cluster Elasticsearch	9200
netsentinel-backend.service	API backend (uvicorn)	8010
netsentinel-ai-engine.service	Moteur de détection (uvicorn)	9000
netsentinel-frontend.service	Interface web (Node)	3000
netsentinel-ai-cycle.timer	Déclencheur du cycle (5 min)	—

Commandes d'exploitation :

```
# État des services
systemctl is-active netsentinel-backend netsentinel-ai-engine \
    netsentinel-frontend netsentinel-elasticsearch

# Journaux d'un service
sudo journalctl -u netsentinel-ai-engine -f

# Prochaine exécution du cycle de détection
systemctl list-timers netsentinel-ai-cycle

# Déclencher un cycle manuellement
curl -X POST http://127.0.0.1:9000/run-once -H "Content-Type: application/json" -d '{}'
```

```
# Santé de la plateforme
curl http://127.0.0.1:8010/api/health
```

9.3 Le timer de détection

C'est la pièce qui rend la plateforme **autonome**. Sans lui, le moteur IA n'analyse rien : il attend un appel qui ne vient jamais.

```
[Timer]
OnBootSec=3min      # premier cycle 3 min après le démarrage
OnUnitActiveSec=5min # puis toutes les 5 minutes
AccuracySec=30s
```

La fenêtre d'analyse étant de 10 minutes (LOOKBACK_MINUTES) pour un cycle toutes les 5 minutes, le recouvrement garantit qu'**aucun événement n'échappe à l'analyse**. Les doublons induits par ce recouvrement sont absorbés par la déduplication.

10 Journal des corrections — 13 juillet 2026

Cette section documente une session de diagnostic qui a rétabli le cœur fonctionnel de la plateforme. Elle a valeur d'enseignement : **la télémétrie était parfaitement saine, et pourtant la plateforme ne détectait rien.**

10.1 Le symptôme

/api/alerts et /api/incidents renvoyaient des listes **vides**, alors qu'Elasticsearch contenait plus de 5 millions de journaux et que les quatre services étaient actifs. Tous les voyants étaient au vert, et rien ne fonctionnait.

10.2 Trois défauts en cascade, chacun masquant le suivant

10.2.1 Défaut 1 — Mauvais port Elasticsearch

ai_engine/.env pointait sur http://79.137.32.27:9201, alors qu'Elasticsearch écoute sur le port **9200**. Chaque cycle de détection échouait en ConnectionRefused **avant même** d'interroger la moindre donnée.

La faute était recopiée dans .env.example, le guide de déploiement, le README des agents et la page Guide utilisateur : la configuration erronée se propageait à chaque nouveau déploiement. **Corrigée à la racine, dans les cinq fichiers.**

10.2.2 Défaut 2 — Chemin d'état inexistant

STATE_DIR pointait sur /home/paul/Bureau/Projects/netsentinel-ai/ai-engine/state — un chemin du **poste de développement**, absent du serveur. La persistance de l'état de déduplication et des modèles ML ne pouvait pas fonctionner. Corrigé vers /home/ubuntu/netsentinel-ai/ai_engine/state.

10.2.3 Défaut 3 — Régression de refactoring, masquée par une erreur silencieuse

Le défaut le plus instructif. backend/server.py (ligne 696) lisait finding.mitre_techniques, un champ **perdu** lors du refactoring de ns_schemas.py vers schemas.py. Chaque POST /api/ai/findings levait donc une AttributeError et renvoyait **500**.

Or le publisher du moteur IA avale les erreurs sans les journaliser :

```
except requests.RequestException:
    # Backend unreachable – skip this cycle, will retry next run
    pass
```

Résultat : le moteur détectait correctement les menaces, le backend les rejetait toutes en erreur 500, et personne n'en savait rien. Aucune trace, aucun symptôme — seulement une liste d'alertes vide.

C'est l'enseignement principal de cette session : *une exception avalée sans journalisation transforme une panne bruyante en panne invisible*. Une simple ligne de log aurait rendu le diagnostic immédiat.

10.3 Vérification après correction

Contrôle	Avant	Après
Cycle de détection	500 (ConnectionRefused)	200 OK
Menaces détectées	0	4 (2 heuristiques + 2 RF)
Findings publiés	0	4
/api/alerts	vide	4 alertes
/api/incidents	vide	1 incident
Index ai-alerts-manual	inexistant	créé
Exécution du cycle	manuelle uniquement	automatique (5 min)

10.4 Corrections livrées

Deux commits poussés sur main (dépôt [eliote-geeks/soutenance_ict_l3](#)) :

- beca545 — *fix(ai): reconnecte le pipeline de detection de bout en bout*
- 03c65cc — *feat(soutenance): telemetrie agents reelle, mode presentation, serveur frontend*

11 Limites connues

Les assumer clairement est un signe de maîtrise. Les dissimuler est le meilleur moyen de se faire piéger.

#	Limite	Portée	Remédiation
1	Le blocage d'IP n'est pas effectif : /api/firewall/block alimente un ensemble en mémoire, sans règle iptables.	La réponse est <i>tracée</i> , pas <i>appliquée</i> .	Invoquer iptables -A INPUT -s <ip> -j DROP (ou ufw deny) depuis la route, avec journal d'audit.
2	L'isolation d'hôte est symbolique : /api/hosts/{id}/isolate change un statut en mémoire.	Aucune action réseau réelle.	Commande d'isolation transmise à l'agent.
3	Le Random Forest est entraîné sur des données synthétiques (150 échantillons générés).	Performances réelles non mesurées.	Réentraîner sur les findings confirmés par l'analyste.
4	État en mémoire perdu au redémarrage : les alertes vivent dans un tampon Python (AI_FINDINGS_BUFFER) autant que dans Elasticsearch.	Le tampon est vidé à chaque redémarrage du backend.	Lire systématiquement depuis Elasticsearch.
5	Services exposés publiquement : Elasticsearch (9200), backend (8010) et moteur IA (9000) écoutent sur 0.0.0.0 sans authentification. Les journaux montrent des robots qui les sondent déjà.	Risque de sécurité réel — paradoxal pour une plateforme de sécurité.	Restreindre à 127.0.0.1 + reverse proxy avec TLS et authentification.

#	Limite	Portée	Remédiation
6	Un seul agent actif sur trois : seul paul_serveur remonte des données ; annie_pc et chelsy_pc sont silencieux.	La démonstration multi-machines n'est pas opérationnelle.	Réinstaller/réapprouver les deux agents.
7	Volumétrie non maîtrisée : ~1,1 Go/jour, saturation du disque estimée sous ~50 jours.	Dettes d'exploitation.	Politique ILM de rétention à 30 jours.
8	Deux lignées de code coexistent (ns_*.py et modules refactorisés).	Source de la régression du défaut 3.	Supprimer la lignée ns_* devenue morte.
9	Cluster à nœud unique (état yellow).	Aucune tolérance de panne.	Hors périmètre pour un projet de laboratoire — à assumer tel quel.

11.1 Les deux priorités avant la soutenance

1. **Rebrancher les agents annie_pc et chelsy_pc** — la supervision multi-machines est un argument central du projet, et une démonstration sur une seule machine l'affaiblit nettement.
2. **Rendre le blocage d'IP effectif** — c'est la différence entre une plateforme qui *détecte* et une plateforme qui *protège*. Le code nécessaire tient en quelques lignes, et l'effet en démonstration est spectaculaire.

12 Ce qu'il faut défendre en soutenance

12.1 Les points forts

1. **La chaîne est complète et réellement opérationnelle** — collecte, stockage, analyse, visualisation, réponse. Ce n'est pas une maquette : la plateforme tourne sur un serveur exposé et analyse plus de 46 millions d'événements réels.
2. **La détection est hybride** — trois familles de détecteurs (règles, non supervisé, supervisé) qui se couvrent mutuellement. La faiblesse de l'une est compensée par les autres.
3. **Les détections sont explicables** — chaque alerte porte une tactique MITRE ATT&CK, un niveau de confiance et une recommandation.
4. **Les alertes sont réelles** — les attaques détectées sont de véritables tentatives d'intrusion, pas un jeu de démonstration. C'est l'argument le plus fort du projet : **le montrer en direct**.
5. **Deux méthodes indépendantes convergent** — l'heuristique SSH et le Random Forest qualifient les mêmes IP en *Credential Access*. Cette convergence est une validation croisée.
6. **L'architecture est modulaire** — stockage interchangeable (Elastic / JSON / PostgreSQL), composants découplés, détection séparée de la décision.
7. **Le déploiement est maîtrisé** — agents installables, enrôlement contrôlé en quatre états, services systemd, cycle autonome.

12.2 Les questions probables du jury — et les réponses

Question	Réponse
« Pourquoi le cluster est-il yellow ? C'est en panne ? »	Non. Sur un nœud unique, Elasticsearch ne peut placer les répliques (une réplique sur la même machine ne protégerait de rien). Les données sont intègres. yellow est l'état normal attendu.
« Votre IA est-elle entraînée sur de vraies données ? »	Le RandomForest : non, sur un jeu synthétique de 1 150 échantillons — c'est assumé et documenté. L'IsolationForest : oui , il apprend sur l'historique réel du réseau (24 h). Et les heuristiques ne dépendent d'aucun apprentissage. La validation vient du terrain : les deux méthodes convergent sur les attaques réelles.
« Bloquez-vous vraiment les attaquants ? »	Aujourd'hui, le blocage est logique : l'IP est marquée, tracée, affichée, mais aucune règle iptables n'est encore posée. C'est identifié comme la première évolution à livrer.

Question	Réponse
« Que se passe-t-il si une attaque dure une heure ? »	La déduplication par signature SHA-256 (fenêtre 60 min) évite de republier la même alerte à chaque cycle. L'analyste voit une alerte, pas douze par heure.
« Comment détectez-vous une attaque inconnue ? »	C'est le rôle de l'IsolationForest : il apprend le comportement normal du réseau et signale ce qui s'en écarte, sans règle préécrite ni étiquetage.
« Un agent peut-il espionner à l'insu de l'admin ? »	Non. Le flux install → enroll → approve → active impose une approbation explicite côté serveur. Sans elle, l'agent ne collecte rien.
« Pourquoi 5 minutes entre deux cycles ? »	Compromis entre réactivité et charge. La fenêtre d'analyse (10 min) recouvre l'intervalle (5 min), donc aucun événement n'est perdu.

12.3 Le déroulé de démonstration recommandé

1. Montrer la page **Overview** : la plateforme est vivante, les compteurs sont réels.
2. Ouvrir **Alerts** : « ces quatre alertes sont de vraies tentatives d'intrusion sur notre serveur, détectées cette nuit ».
3. Ouvrir une alerte : montrer la tactique MITRE, la confiance, la recommandation.
4. Ouvrir **Incidents** : montrer que les alertes se corrélaient en une seule histoire d'attaque.
5. Déclencher un cycle en direct (POST /run-once) et montrer le résultat JSON : détecteurs, findings, IP bloquées.
6. Montrer la page **Agents** : le contrôle d'enrôlement.
7. Conclure sur les limites assumées et la feuille de route.

13 Annexes

13.1 Annexe A — Référence complète de l'API (42 routes)

Santé et vue d'ensemble

Méthode	Route	Rôle
GET	/api/health	Santé de la plateforme (Elastic, IA, stockage)
GET	/api/overview	Indicateurs du tableau de bord
GET	/api/pipeline	Santé de la chaîne de traitement
GET	/api/stream	Flux d'événements temps réel

Sécurité

Méthode	Route	Rôle
GET	/api/alerts	Liste des alertes
POST	/api/alerts/{id}/acknowledge	Acquitter une alerte
GET	/api/incidents	Incidents corrélés
GET	/api/logs	Journaux bruts
GET	/api/hosts	Hôtes supervisés
POST	/api/hosts/{id}/isolate	Isoler un hôte
POST	/api/firewall/block	Bloquer une adresse IP

Intelligence artificielle

Méthode	Route	Rôle
GET	/api/ai/status	État du moteur IA
GET	/api/ai/findings	Findings produits
POST	/api/ai/findings	Ingestion d'un finding (appelée par le moteur)
GET	/api/ai/recommendations	Recommandations de remédiation
GET	/api/ai/attack-knowledge-base	Base de connaissance des attaques
GET	/api/model	Métadonnées des modèles
GET	/api/predictions	Prédictions
POST	/api/chatbot/ask	Assistant intégré

Agents

Méthode	Route	Rôle
GET / POST	/api/agent/enrollment-tokens	Lister / créer un jeton
POST	/api/agent/enrollment-tokens/{id}/revoke	Révoquer un jeton

Méthode	Route	Rôle
POST	/api/agent/enroll	Enrôlement d'un agent
GET	/api/agent/instances	Agents enrôlés
POST	/api/agent/instances/{id}/approve	Approuver
POST	/api/agent/instances/{id}/reject	Rejeter
POST	/api/agent/instances/{id}/disable	Désactiver
POST	/api/agent/instances/{id}/actions	Commander un agent
POST	/api/agent/checkin	Récupération de configuration
POST	/api/agent/heartbeat	Signal de vie
GET	/api/agent/installers/sources/{id}	Télécharger un installateur

Inventaire, administration et divers

Méthode	Route	Rôle
GET / POST	/api/profiles	Profils métier
GET / POST	/api/assets	Actifs supervisés
POST	/api/profile-assets	Rattachement actif ↔ profil
GET	/api/scope, /api/scope/options	Périmètre de supervision
POST	/api/admin/session	Session administrateur
POST	/api/reports/export	Export de rapport
POST	/api/tickets	Création de ticket

Moteur IA (port 9000)

Méthode	Route	Rôle
GET	/health	Santé du moteur
GET	/status	Seuils et paramètres actifs
POST	/run-once	Exécuter un cycle de détection

Assistant de configuration

/api/setup/status, /api/setup/test-connection, /api/setup/current-config, /api/setup/reset, /api/setup/complete

13.2 Annexe B — Variables d'environnement

Elasticsearch (backend et moteur IA)

Variable	Valeur en production
ELASTICSEARCH_URL	http://127.0.0.1:9200
ELASTICSEARCH_USERNAME	elastic
ELASTICSEARCH_PASSWORD	(secret)
ELASTICSEARCH_API_KEY	(secret)

Variable	Valeur en production
ELASTICSEARCH_VERIFY_TLS	false

Backend

Variable	Rôle
NETSENTINEL_STORAGE_BACKEND	elastic json postgresql demo
NETSENTINEL_TELEMETRY_BACKEND	elastic json demo
DATABASE_URL	DSN PostgreSQL (si stockage postgresql)
ADMIN_API_SECRET	Secret de session admin — ne jamais exposer au frontend
AGENT_ELASTIC_API_KEY	Clé remise aux agents
CORS_ORIGINS	Origines autorisées

Moteur IA — seuils de détection

Variable	Valeur	Signification
LOOKBACK_MINUTES	10	Fenêtre d'analyse
SSH_FAILURE_THRESHOLD	5	Échecs SSH avant alerte
DNS_ANOMALY_THRESHOLD	8	Erreurs DNS avant alerte
PORT_SCAN_DISTINCT_PORT_THRESHOLD	6	Ports distincts avant alerte
FINDING_SUPPRESSION_MINUTES	60	Fenêtre de déduplication
ML_HISTORY_HOURS	24	Historique d'apprentissage
ML_BUCKET_MINUTES	15	Granularité des fenêtres
ML_MIN_SAMPLES	10	Échantillons minimum
ML_CONTAMINATION	0.12	Proportion d'outliers attendue
ML_RANDOM_STATE	42	Graine (reproductibilité)
STATE_DIR	./state	Persistance (dédup + modèles)

13.3 Annexe C — Arborescence du projet

```

netsentinel-ai/
├── agent/                                # Agent installable (Linux + Windows)
│   ├── install-linux.sh
│   ├── install-windows.ps1
│   └── ns_agent_runtime.py
├── ai_engine/                            # Moteur de détection IA
│   ├── app.py                           # API FastAPI (:9000)
│   ├── cycle.py                         # Orchestration du cycle en 8 étapes
│   ├── features.py                     # Agrégation des 8 features
│   ├── heuristics.py                   # Les 5 détecteurs à base de règles
│   ├── ml_models.py                   # IsolationForest + RandomForest
│   └── prevention.py                   # Décision de blocage (+ liste blanche)

```

```

├── publisher.py          # Déduplication SHA-256 + publication
├── train_random_forest.py# Entraînement du modèle supervisé
├── state/               # Modèles persistés + état de dédup
├── backend/             # API métier
│   ├── server.py        # 42 routes REST (:8010)
│   ├── elastic.py       # Accès Elasticsearch
│   ├── schemas.py       # Contrats de données (Pydantic)
│   ├── agents.py        # Cycle de vie des agents
│   └── ns_storage.py     # Stockage interchangeable
├── frontend/           # Interface React (:3000)
│   ├── src/pages/       # Les 18 pages SOC
│   ├── src/components/
│   └── server.js        # Serveur de production (Node)
├── packaging/          # Construction des paquets .deb / .msi
├── docs/               # Documentation
└── tests/

```

13.4 Annexe D — Les alertes réelles du 13 juillet 2026

Sévérité	Titre	IP source	Hôte	Tactique MITRE	Détecteur
critical	Random Forest: Bruteforce detected	45.148.10.239	vps-8e515079	Credential Access	RandomForest
critical	Random Forest: Bruteforce detected	135.181.107.18	vps-8e515079	Credential Access	RandomForest
critical	SSH brute force suspected	135.181.107.18	vps-8e515079	Credential Access	Heuristique
high	SSH brute force suspected	45.148.10.239	vps-8e515079	Credential Access	Heuristique

Incident corrélé : INC-00001 — *Credential Access from 45.148.10.239* — sévérité critical, statut active.

Adresses soumises au blocage : 135.181.107.18, 45.148.10.239, 77.90.185.20.

À noter : les deux adresses les plus agressives sont détectées **deux fois chacune**, par deux détecteurs indépendants — une heuristique déterministe et un modèle d'apprentissage supervisé. Cette convergence, sur des attaques réelles et non simulées, constitue la meilleure validation empirique du moteur hybride.

13.5 Annexe E — Environnement technique

Composant	Version
Elasticsearch	9.4.2
Python	3.13.3
Node.js	20.20.1
Elastic Beats	8.19.16
FastAPI / Uvicorn	—
scikit-learn	1.8.0
React	18

Dépôt : `git@github.com:eliote-geeks/soutenance_ict_l3.git` (branche main)

Document généré le 13 juillet 2026. Toutes les mesures et alertes citées proviennent d'observations réelles sur le serveur de production 79.137.32.27.